

## Etape 2 livree, et pourquoi 1b et 3 ne le sont pas

Force Tracker — 12/09/2026, ft - v1194. Sixieme document de la serie (DOUANE -> ARCHI -> PLAN -> PHASE0A-ETAPE1A -> VALIDATION-IPHONE -> celui-ci). Michel a valide la phase 0a **sur iPhone** et donne le feu vert pour **1b, 2 et 3**, meme methode que 1a. **Une seule des trois est livree**, et c'est sa propre consigne qui l'a decide.

### LA CONSIGNE QUI DECIDE DE CETTE VERSION

« Si une regression ou une divergence reelle apparait pendant l'extraction, **mesure-la et arrete-toi** avant de la 'corriger au passage'. »

**La divergence est apparue AVANT la premiere ligne de code** — et elle ne porte pas sur le code, elle porte sur le **perimetre ecrit dans le plan**.

### 1. Le perimetre : ce que le plan annonce, ce que la mesure donne

| etape                                | perimetre du plan | perimetre MESURE                                                                                                                                                                                                                                           | fait ?        |
|--------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>1b</b> — la « forme aliment »     | 5 sites           | <b>19 lignes</b> portent le SEUL champ portionWeightG ( <i>recompte ici</i> ). Le balayage donne <b>au moins 15</b> lecteurs/ecrivains de la forme aliment, en <b>3 ecritures differentes</b> ( <i>mesure a la main, PAS recomptee par ce generateur</i> ) | <b>non</b>    |
| <b>2</b> — le pour-100 g derive      | 3 sites           | <b>3</b> — <i>le seul juste</i>                                                                                                                                                                                                                            | <b>livree</b> |
| <b>3</b> — « quantite utilisable ? » | 6 sites           | <b>7 ecritures</b> en 2 formes (5 + 2), <b>+ 9 lignes</b> de la moitie PORTIONS, jamais comptee                                                                                                                                                            | <b>non</b>    |

**Et ce ne sont pas des ecarts de comptage : les DEFAUTS divergent d'un site a l'autre.** C'est ce qui interdit une extraction mecanique — choisir une valeur ne serait pas un rangement, ce serait une **decision** qui change des lignes deja enregistrees :

| champ          | ce qu'on trouve selon le site     |
|----------------|-----------------------------------|
| q              | tantot 0, tantot null             |
| portionWeightG | 0, null, ou la cle est absente    |
| origine        | null, 'utilisateur', ou 'reprise' |

L'instantane tools/instantane\_1b23.js **fige ces trois ecarts expres** : il est ce qui empeche de les harmoniser « au passage » sans s'en apercevoir.

## 2. Ce qui est livre : un seul propriétaire du pour-100 g derive

Trois endroits retapaient la meme algebre — totaux x 100 / masse, macro par macro, arrondie a la decimale.

```
// AVANT – la meme algebre, retapee a trois endroits

// _provFood, branche GRAMMES
const f=100/q;
p.per100={kcal:_per100d1((+vals.kcal||0)*f),
  prot:_per100d1((+vals.prot||0)*f),
  carbs:_per100d1((+vals.carbs||0)*f),
  fat:_per100d1((+vals.fat||0)*f)};

// saveEditFood
const f=100/masse;
e.per100={kcal:_per100d1((+e.kcal||0)*f), prot:_per100d1((+e.prot||0)*f),
  carbs:_per100d1((+e.carbs||0)*f), fat:_per100d1((+e.fat||0)*f)};

// _provFood, branche PORTIONS
const f=100/masse;
p.per100={kcal:_per100d1((+vals.kcal||0)*f),
  prot:_per100d1((+vals.prot||0)*f),
  carbs:_per100d1((+vals.carbs||0)*f),
  fat:_per100d1((+vals.fat||0)*f)};

// APRES – un seul propriétaire, et il sait DIRE QU'IL NE SAIT PAS

function _per100Derive(vals, masse){
  const m=+masse||0;
  if(!(m>0)) return null; // <- ce null EFFACE un pour-100 g orphelin (R29)
  const f=100/m;
  return {kcal:_per100d1((+vals.kcal||0)*f), prot:_per100d1((+vals.prot||0)*f),
    carbs:_per100d1((+vals.carbs||0)*f), fat:_per100d1((+vals.fat||0)*f)};
}

// les 3 appelants
p.per100=_per100Derive(vals, q); // grammes
if(!p.per100){ const d=_per100Derive(vals, masse); if(d) p.per100=d; } // portions
const d=_per100Derive(e, (+e.q||0)*(+e.portionWeightG||0)); // edition
if(d) e.per100=d; else if(e.per100) delete e.per100;
```

### LA DUPLICATION AVAIT DEJA COUTE, SUR CES LIGNES-LA

En **ft-v1188**, le passage de `Math.round` a `_per100d1` a du etre pose sur **deux** d'entre elles — apres l'avoir ete sur **7 autres portes** en **ft-v1170**. C'est la famille **\$59** du depot (*la porte jumelle*), a sa 8<sup>e</sup> occurrence.

*La question n'etait pas de savoir si la 3<sup>e</sup> copie serait oubliee, mais quand.*

## 3. Le point de conception : ce qu'on factorise est l'INTENTION, pas la ressemblance

A six lignes du nouveau propriétaire vit une fonction qui porte **exactement la meme algebre**. On est tente de l'absorber. **Elle est lailsee dehors.**

```
// LAISSEE DEHORS, EXPRES – meme algebre, autre metier

function _per100SuitLaPortion(av){ // elle VERIFIE : « ce pour-100 g
  if(!av.per100) return true; // venait-il d'une portion ? »
  const m=(+av.q||0)*(+av.pw||0); // ... et non « quelle est sa valeur ? »
  if(!(m>0)) return false;
  const f=100/m, ok=(a,b)=>Math.abs((+a||0)-(+b||0))<=0.6; // <- sa tolerance
  return ok(av.per100.kcal,(+av.kcal||0)*f) && ...
}
```

Elle **verifie** (« ce pour-100 g venait-il d'une portion ? », a 0,6 pres), elle ne **derive** pas. La fondre dans le propriétaire changerait **son metier**, pas son code. Un temoin fige qu'elle est toujours la, avec sa tolerance.

### ET LE PROPRIETAIRE SAIT DIRE QU'IL NE SAIT PAS

Masse nulle, negative, illisible -> il rend null. **Ce null n'est pas decoratif** : c'est lui qui, dans `saveEditFood`, **EFFACE** un pour-100 g devenu orphelin quand la personne retire le poids de sa portion. **R29** — un null ne se remplace jamais par un default.

#### 4. L'etape 3 en detail : le plan comptait la moitie de la question

Le « 6 » du plan etait presque juste sur la regle stricte : elle est ecrite 7 fois, en 2 formes.

| forme                   | combien | ce qu'elle accepte                            |
|-------------------------|---------|-----------------------------------------------|
| +q>0 && (!u    u==='g') | 5       | les grammes <b>seuls</b> (et l'unite absente) |
| ...    u==='portion'    | 2       | les grammes <b>et</b> les portions            |

**CE QUE LE PLAN A RATE : LA MOITIE PORTIONS**

u==='portion' apparait **9 fois** de plus, sur des lignes qui decident elles aussi de ce qu'on fait de la quantite. **Elle a ete ajoutee en ft-v1183 / ft-v1186 et jamais reintegree a l'inventaire.**

Total reel : **16 decisions** sur l'unite, pas 6.

**ET J'AI FAILLI LIVRER ICI LE MEME GENRE DE CHIFFRE QUE CELUI QUE JE REPROCHE AU PLAN**

Ma premiere redaction annoncait « ~17 sites, au moins 3 ecritures de la regle », en citant \_provFood @1232 contre @1311 comme **deux ecritures non equivalentes.**

**Relu ligne a ligne avant de pousser : c'est faux.** @1311 ne pose pas la meme question — elle interroge \_afRef (l'etat de l'ecran), pas \_afSrc (la provenance de l'aliment) — et son === strict y est **necessaire** : \_afRef . u vaut ' ' dans l'etat « portions », donc un ! u permissif ferait tomber une portion dans la branche grammes.

**Un chiffre rond se verifie ligne a ligne — surtout quand il sert a demontrer qu'un autre chiffre etait faux.**

## 5. Pourquoi le compteur s'est trompé (nouvelle famille §63)

```
// POURQUOI LE COMPTEUR S'EST TROMPE — addFoodEntry, la porte la plus utilisee

const _e=Object.assign({date:_journalJourActif(), meal:_afMeal, name:name.slice(0,80),
                        kcal, prot, carbs, fat, ts:Date.now()}, // <- raccourci ES6
                        _provFood({kcal,prot,carbs,fat}));

S.foodLog.push(_e);

// Il n'y a PAS UN SEUL « kcal: » dans cette ligne.
// Un motif /kcal\s*/ ne la voit pas — et il ne signale rien, puisqu'il trouve les autres.
```

Même cause pour l'export CSV de setup.js : **13 colonnes** aux noms **français** (quantite, unite, portion\_label...), avec sa liste figée à part. Aucun motif pensé pour le schéma interne ne peut la voir.

### LA FAMILLE, ÉCRITE DANS `BUGS.md` §63

**Un motif de recherche qui suppose une SYNTAXE ne compte pas les endroits : il compte les endroits écrits comme on les imaginait.**

**Pourquoi c'est couteux** : le chiffre sert ensuite à **dimensionner un chantier**. Une extraction « de 6 sites » qu'on découvre à 16 en cours de route se termine de deux façons, toutes deux mauvaises — on livre à moitié, ou on **elargit en silence** un périmètre que personne n'a validé. *Un inventaire faux ne se manifeste pas comme une erreur : il se manifeste comme un chantier qui déborde.*

**Ce qui l'attrape** : compter **deux fois, autrement** (par motif de texte **et** par ce que la fonction FAIT — ici « qui écrit dans S.foodLog ? », qui trouve les **3** écrivains quelle que soit leur syntaxe) ; croiser avec les appelants (*la porte principale d'un écran ne peut pas être absente d'un inventaire de cet écran*) ; et **dire le chiffre avec sa méthode** — « 7 lignes correspondent au motif X, recompte à chaque génération » se vérifie ; « il y a 6 sites » ne se vérifie pas, parce qu'on ne sait même pas ce qui a été compté.

Soeur de §58 (vérifier la fonction n'est pas vérifier l'appel) et de §61 (un outil de mesure tronqué ressemble à un code sans défaut) : les trois disent la même chose à trois endroits — **l'instrument fait partie de la mesure**.

## 6. Comment on sait que rien n'a bougé

| preuve                                                         | resultat                                                                                                                                      |
|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Instantane tools/instantane_1b23.js (11 sondes), avant / après | <b>identique octet pour octet</b> , même sha256 ace2a744dc89e6ec — <b>les sondes de 1b et 3 comprises</b> , donc l'extraction n'a pas débordé |
| Passe parcours (bloc CCXCI, +15 témoins)                       | <b>3564 / 3564</b> — et le total <b>prédit</b> était 3549 + 15 = 3564 : il colle, donc la passe n'est pas tronquée (§61)                      |
| Calculs / muscles / croises / dates                            | <b>339 / 241 / 50 / 9</b> , tous verts                                                                                                        |
| Données classées face à Milo                                   | <b>aucun trou nouveau</b> (les 2 connus restent inchangés)                                                                                    |
| Contrôle négatif                                               | <b>8 mutations, TOUTES mordent</b> , chacune sur son témoin                                                                                   |
| Déploiement (R18)                                              | run # <b>1091</b> , success à 08:02:02 UTC                                                                                                    |

### UNE MUTATION A TUE MA SONDE AU LIEU DE LA FAIRE ROUGIR

Le témoin des 4 macros écrivait `Object.keys(_per100Derive(...))`. Avec le propriétaire mute pour rendre null, ça **leve** — l'évaluateur entier est rejeté, et le **bloc disparaît de la passe sans qu'elle rougisse**.

Rendu défensif, la mutation fait désormais **7 rouges nommes**. *Un témoin qui MEURT ressemble à un témoin qui passe* — c'est §61 en miniature, et la raison est écrite à l'endroit exact.

## 7. Ce qui attend une decision de Michel

Deux questions, et elles ne sont pas techniques. Elles sont ecrites dans docs/JOURNAL - DE - TEST .md pour ne pas disparaitre avec la session (R27).

| question                                                                                                                                          | ce qui en depend                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. 1b et 3 se font-elles sur leur <b>perimetre reel</b> (un chantier ~3x plus gros que le plan), ou <b>re-decoupees</b> en morceaux plus petits ? | le decoupage du chantier, et le nombre de versions avant le hub                                                                                                                                                      |
| 2. Les <b>defauts divergents</b> s'harmonisent-ils ?                                                                                              | si <b>oui</b> , c'est une <b>decision produit</b> : elle change des lignes deja enregistrees. Si <b>non</b> , la « forme aliment » unique devra les <b>transporter</b> — ce qui limite beaucoup l'interet de l'etape |

### Hors perimetre, et ca le reste

| sujet                                              | etat                                                                                                                                     |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Hub (etape 4) et douane (etape 5)                  | apres 1b et 3 — consigne explicite de Michel                                                                                             |
| S. savedFoods perdu entre deux onglets             | <b>ouvert</b> . Le correctif evident est <b>faux</b> : une union par nom ferait <b>ressusciter une etoile retiree</b> . Decision produit |
| L'ecart <b>48,3</b> vs <b>48</b> sur la meme fiche | <b>ouvert</b> . Corriger changerait une valeur enregistree — ce n'est pas une extraction                                                 |
| Historique abime, migrations, Milo, seances        | <b>non touches</b>                                                                                                                       |

#### LIMITE DITE PLUTOT QUE TUE

Le proxy de ce conteneur refuse `github.io` (403), donc **je ne peux pas lire le sw.js reellement servi**. La verification s'arrete a l'API GitHub : *le run est vert, l'app affichant ft-v1194 reste a confirmer par Michel dans « A propos »*.

Document genere depuis le code servi — **tous les decomptes cites sont recomptes a chaque generation** (1 proprietaire, 4 appels, 5 + 2 ecritures de la regle, 9 lignes de la moitie portions, 13 colonnes CSV, 3 ecrivains). *Un document qui cite un decompte sans le verifier refait exactement l'erreur qu'il raconte*. Source : `tools/gen_etape2_perimetre_pdf.py`.